

```

import {
  addalumno,
  getalumnosCollection,
  deletealumnoCollection,
  getalumnoCollection,
  updatealumnoCollection,
} from "../firebase.js";

/**
 * Función para refrescar la tabla después de cualquier acción
 */
async function refrescarTabla() {
  await mostraralumnosEnHTML();
}

window.miModal = async function (idModal, idalumno = "") {
  try {
    // 1. Limpieza de seguridad
    const existing = document.getElementById(idModal);
    if (existing) {
      const ins = bootstrap.Modal.getInstance(existing);
      if (ins) ins.dispose();
      existing.remove();
    }
    document.querySelectorAll('.modal-backdrop').forEach(el => el.remove());

    // 2. Cargar el PHP
    let url = "";
    switch (idModal) {
      case "agregaralumnoModal": url = "modales/modalAdd.php"; break;
      case "detallealumnoModal": url = "modales/modalDetalles.php"; break;
      case "editaralumnoModal": url = "modales/modalEditar.php"; break;
      case "eliminaralumnoModal": url = "modales/modalDelete.php"; break;
      default: return;
    }

    const response = await fetch(url);
    const text = await response.text();

    // 3. Convertir texto a elementos y FILTRAR solo el DIV del modal
    const parser = new DOMParser();
    const htmlDoc = parser.parseFromString(text, 'text/html');

    // Esto busca el div.modal real ignorando comentarios y textos
    const modalElement = htmlDoc.querySelector('.modal');

    if (!modalElement) {
      console.error("No se encontró un div con clase 'modal' en el archivo cargado.");
      return;
    }

    // 4. Inyectar y mostrar
    document.body.appendChild(modalElement);
  }
};

```

```

const myModal = new bootstrap.Modal(modalElement);
myModal.show();

// 5. Cargar datos según corresponda
if (idModal === "detallealumnoModal") {
    await cargarDetallealumno(idalumno);
} else if (idModal === "editaralumnoModal") {
    await getalumnoUpdateCollection(idalumno);
} else if (idModal === "eliminaralumnoModal") {
    const btn = modalElement.querySelector("#confirmDeleteBtn");
    if (btn) btn.onclick = async () => { await eliminaralumno(idalumno);
myModal.hide(); };
}

} catch (error) {
    console.error("Error al abrir modal:", error);
}
};

async function mostraralumnosEnHTML() {
    try {
        const queryCollection = await getalumnosCollection();
        const tablaalumnos = document.querySelector("#tablaalumnos tbody");
        if (!tablaalumnos) return;

        tablaalumnos.innerHTML = "";

        queryCollection.forEach((doc) => {
            const alumno = doc.val();
            const id = doc.key;
            const fila = document.createElement("tr");
            fila.id = id;
            fila.innerHTML = `
                <td>${alumno.curso}</td>
                <td>${alumno.nombre}</td>
                <td>${alumno.dni}</td>
                <td>${alumno.obs}</td>
                <td>
                    <a title="Ver detalles" href="#"
onclick="window.miModal('detallealumnoModal','${id}')" class="btn btn-success
btn-sm">
                        <i class="bi bi-binoculars"></i>
                    </a>
                    <a title="Editar" href="#"
onclick="window.miModal('editaralumnoModal','${id}')" class="btn btn-warning
btn-sm">
                        <i class="bi bi-pencil-square"></i>
                    </a>
                    <a title="Eliminar" href="#"
onclick="window.miModal('eliminaralumnoModal','${id}')" class="btn btn-danger
btn-sm">
                        <i class="bi bi-trash"></i>
                    </a>
                </td>
            `;
        });
    }
}

```

```

        </td>
    `;
    tablaalumnos.appendChild(fila);
    });
} catch (error) {
    console.error("Error al obtener los alumnos:", error);
}
}

/**
 * Función para mostrar alertas (Usando iziToast)
 * Movida arriba para que esté disponible cuando se necesite.
 */
window.mostrarAlerta = function ({ tipoToast, mensaje }) {
    if (typeof iziToast === 'undefined') {
        console.warn("iziToast no está cargado, usando alert normal:", mensaje);
        alert(mensaje);
        return;
    }

    if (tipoToast === "success") {
        iziToast.success({
            timeout: 5000,
            icon: "bi bi-check-circle-fill",
            title: "¡Éxito!",
            message: mensaje,
            position: "topRight"
        });
    } else if (tipoToast === "error" || tipoToast === "warning") {
        iziToast.error({
            timeout: 5000,
            icon: "bi bi-x-circle-fill",
            title: "¡Error!",
            message: mensaje,
            position: "topRight"
        });
    }
};

// ... resto de tus funciones (addNuevoalumno, etc.)

/**
 * CREATE: Agrega un nuevo alumno
 */
window.addNuevoalumno = async function (event) {
    event.preventDefault();
    const formulario = document.querySelector("#formularioalumno");
    const formData = new FormData(formulario);
    const dataJSON = Object.fromEntries(formData.entries());

    try {
        await addalumno(dataJSON.curso, dataJSON.nombre, dataJSON.dni,
dataJSON.obs, -1); //-1 sin id de huella asigando
    }

```

```

    formulario.reset();

    const modalElt = document.getElementById("agregaralumnoModal");
    bootstrap.Modal.getInstance(modalElt).hide();

    window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Alumno
registrado con éxito!" });
    await refrescarTabla();
  } catch (error) {
    console.error("Error al añadir:", error);
  }
};

/**
 * READ ONE: Detalles del alumno
 */
async function cargarDetallealumno(id) {
  try {
    const alumnoDoc = await getalumnoCollection(id);
    if (alumnoDoc.exists()) {
      const data = alumnoDoc.val();
      const contenedor = document.querySelector("#detallealumnoContenido
ul");

      if (!contenedor) return;

      contenedor.innerHTML = `
        <li class="list-group-item"><b>Curso:</b> ${data.curso ||
"N/A"}</li>
        <li class="list-group-item"><b>Apellido y Nombre:</b>
${data.nombre || "N/A"}</li>
        <li class="list-group-item"><b>DNI:</b> ${data.dni ||
"N/A"}</li>
        <li class="list-group-item"><b>Obs:</b> ${data.obs ||
"N/A"}</li>
      `;
    }
  } catch (error) {
    console.error("Error al cargar detalles:", error);
  }
}

/**
 * UPDATE: Cargar datos en el formulario de edición
 */
async function getalumnoUpdateCollection(id) {
  try {
    const alumnoDoc = await getalumnoCollection(id);
    if (alumnoDoc.exists()) {
      const data = alumnoDoc.val();
      document.querySelector('[name="idalumno"]').value = id;
      document.querySelector('[name="curso"]').value = data.curso;
      document.querySelector('[name="nombre"]').value = data.nombre;
      document.querySelector('[name="dni"]').value = data.dni;
      document.querySelector('[name="obs"]').value = data.obs;
    }
  }
}

```

```

    }
  } catch (error) {
    console.error("Error al obtener datos para editar:", error);
  }
}

/**
 * UPDATE: Guardar cambios del alumno
 */
window.actualizaralumno = async function (event) {
  event.preventDefault();
  const formulario = document.querySelector("#formularioalumnoEdit");
  const formData = new FormData(formulario);
  const { idalumno, ...datosNuevos } = Object.fromEntries(formData.entries());

  try {
    await updatealumnoCollection(idalumno, datosNuevos);

    const modalElt = document.getElementById("editaralumnoModal");
    bootstrap.Modal.getInstance(modalElt).hide();

    window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Alumno actualizado!" });
    await refrescarTabla();
  } catch (error) {
    console.error("Error al actualizar:", error);
  }
};

/**
 * DELETE: Borrar alumno QUEDA DESABILITADA EL QUE BORRA ES EL ESP DE ESTA
MANERA NOS ASEGURAMOS
 * QUE EL BORRADO SE HAGA EN EL ESP, VA A SEGUIR APARECIENDO LISTADO PARA BORRAR
 */
async function eliminaralumno(id) {
  try {
    await deletealumnoCollection(id);
    window.mostrarAlerta({ tipoToast: "success", mensaje: "Alumno eliminado correctamente" });
    await refrescarTabla();
  } catch (error) {
    console.error("Error al borrar:", error);
    window.mostrarAlerta({ tipoToast: "error", mensaje: "Error al eliminar" });
  }
}

/**
 */
async function solicitarBorradoHuella(id) {
  try {
    // En lugar de borrar, marcamos el campo obs
    await marcarParaBorrar(id);

    window.mostrarAlerta({
      tipoToast: "success",

```

```
        mensaje: "Solicitud de borrado enviada al sensor"
    });

    // El refrescarTabla hará que el alumno pase de la lista normal
    // a la lista de "Alumnos a Borrar" que creamos antes
    await refrescarTabla();

} catch (error) {
    console.error("Error al solicitar borrado:", error);
    window.mostrarAlerta({
        tipoToast: "error",
        mensaje: "No se pudo marcar para borrar"
    });
}
}
```

```
// Inicialización
window.addEventListener("DOMContentLoaded", mostraralumnosEnHTML);
```